

FINAL VERSION 3.0

My Indian ETF Robo-Advisor

End-to-End System Architecture & Workflow Manual

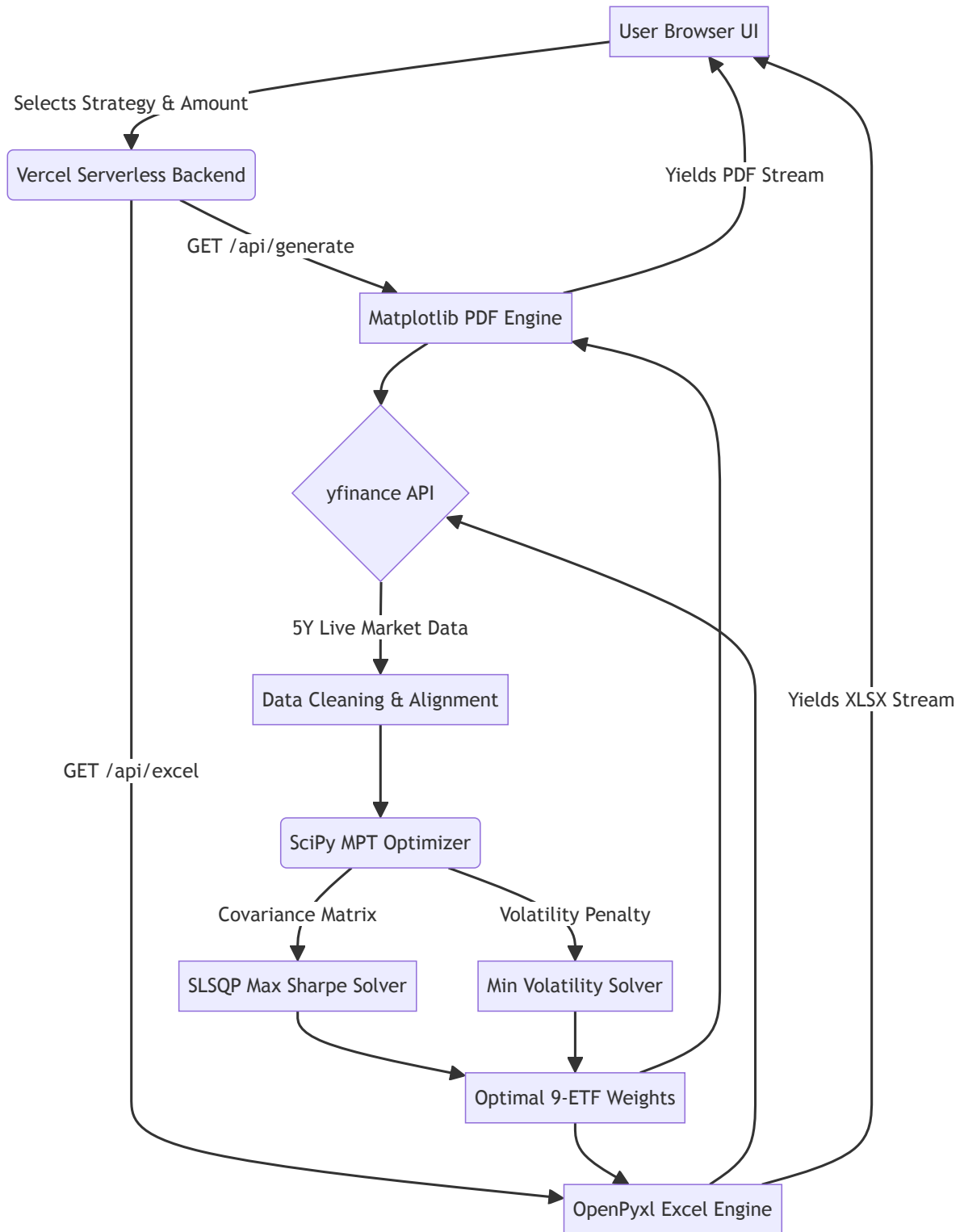
Documenting the full-stack integration: Vercel, SciPy, Matplotlib, and OpenPyxl.

1. Executive Summary

This document outlines the final production architecture of my ETF Portfolio Optimization engine. After building the core Modern Portfolio Theory (MPT) engine in Python, I successfully scaled it into a full-stack, automated **Robo-Advisor**. The platform now features a glassmorphism web interface that connects to a Vercel serverless backend, dynamically generating institutional-grade PDF and Excel reports on the fly.

2. Complete System Workflow Diagram

This flowchart illustrates the end-to-end data pipeline from user input to final report delivery:



3. Serverless Backend & Dynamic Reporting

3.1 Matplotlib PDF Generation

Instead of relying on third-party SaaS services for reporting, I built a bespoke PDF generator using `matplotlib.backends.backend_pdf`. The backend plots the Efficient Frontier, Correlation Heatmaps, and 12-Month Rolling Return charts completely in memory (headless), streams them directly to a PDF buffer, and serves it back to the client.

3.2 Excel Portfolio Wealth Model

For investors wanting to perform their own financial modeling, I integrated `openpyxl` to dynamically construct an Excel workbook. It takes the user's exact investment amount and chosen MPT strategy and builds a three-statement **Portfolio Wealth Model**:

- **Portfolio Beta Calculator:** Calculates live Weighted Beta against the Nifty 50.
- **10-Year Projection Schedule:** Forecasts compounding wealth based on MPT expected returns.
- **Cash Flow Schedule:** Models expected annual dividend yields for the basket.

4. The 9-ETF MPT Optimizer

At the core of the backend is the `scipy.optimize.minimize` algorithm. It ingests 5 years of daily returns across a highly diversified 9-ETF universe (encompassing Indian Equities, US Equities, Gold, Silver, and Government Securities).

The optimizer uses the **SLSQP (Sequential Least Squares Programming)** method to maximize the Sharpe ratio, strictly enforcing boundaries so no single asset exceeds a 45% weight (preventing over-concentration) and the weights sum perfectly to 1.

```
# Core Optimization Logic (SciPy SLSQP)
def maximize_sharpe(mean_returns, cov_matrix, risk_free_rate):
    num_assets = len(mean_returns)
    args = (mean_returns, cov_matrix, risk_free_rate)

    # Target function to minimize (Negative Sharpe)
    def neg_sharpe(weights, mean_returns, cov_matrix, risk_free_rate):
        p_ret = np.sum(mean_returns * weights) * 252
        p_vol = np.sqrt(np.dot(weights.T, np.dot(cov_matrix, weights))) * r
        return -(p_ret - risk_free_rate) / p_vol

    constraints = ({'type': 'eq', 'fun': lambda x: np.sum(x) - 1})
    bounds = tuple((0.0, 0.45) for asset in range(num_assets))

    result = minimize(neg_sharpe, num_assets*[1./num_assets,], args=args,
                      method='SLSQP', bounds=bounds, constraints=constraint
    return result.x
```

End of Final Architecture Manual